TestSprite

Jun 17, 2026

Testing Report

Key Insights From TestSprite's Autonomous AI Testing Agent

Report Contents

40	71.4%	15	3h
Test Runs	Pass Rate	Issues	Saved By TestSprite

This report provides key insights from TestSprite's AI-powered testing. For questions or customized needs, contact us using Calendly or join our Discord community.

Table of Contents

Executive Summary

1 High-Level Overview

OVERVIEW	
Total APIs Tested	0 APIs
Base URL	https://orin-three.vercel.app/
Pass / Fail / Blocked	Backend (endpoint): 0 Passed / 0 Failed Frontend: 25 Passed / 10 Failed / 5 Blocked

2 Key Findings

Test Summary

The executable subset is moderately healthy but not production-stable: 19 tests passed, 6 failed, and 4 were blocked. Excluding the blocked cases, the pass rate is 76% (19/25), which pulls the overall quality down despite strong coverage of auth, notifications, pricing, and several opportunity flows. The main reliability issues are concentrated in AI chat, public verified-profile rendering, proof-card evidence/share flows, and skill-gap analysis. The blocked tests indicate some upstream product states were unavailable, so they are excluded from the score but still reduce end-to-end coverage and confidence in the user experience.

What could be better

Several high-priority user journeys are broken or incomplete. The AI chat surface repeatedly returns “Something went wrong. Please try again.”, blocking coaching notes, learning paths, and artifact review flows. Public profile pages render only “No public proofs yet” with zeroed counters, so recruiter-visible verified proof cards and evidence are missing. Proof-card evidence and sharing are also incomplete: the share API returns {"error":"Invalid request body"}, and proof detail pages omit sources/evidence sections. Skill-gap analysis fails with “Unexpected token '<', \"<!DOCTYPE \"... is not valid JSON,” which strongly suggests an API routing or rewrite problem.

Recommendations

Start with the shared backend/data-contract failures, because they affect multiple high-value features. Fix the AI chat API so it returns a real assistant response instead of the generic error, then verify portfolio analysis, coaching notes, and artifact review. Next, inspect the public-profile and proof-card data pipeline: confirm production environment variables, API URLs, and the proof evidence payload so verified cards, sources, and confidence fields render on the public page. Finally, repair the skill-gap analysis endpoint/rewrite so it returns JSON instead of HTML. After each fix, rerun the relevant E2E tests to confirm the exact titles listed in the suite now pass.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
orin-three.vercel.app	40	25 Pass / 10 Fail / 5 Blocked

Note
The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

Orin-Three.Vercel.App Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Contact and Support: Reject an invalid contact email	A visitor should not be able to submit the contact form with an invalid email address. The page should show an error and prevent the inquiry from being sent.	Low	Passed
Authentication and Account Access: Create an Orin account	A new visitor can register for Orin and reach the authenticated area. The account creation flow should complete successfully and leave the user signed in.	High	Passed
AI Chat and Artifact Experience: Stop a running AI chat generation	A user cancels an in-progress chat generation before it finishes. The test verifies that generation stops and the conversation reflects the interruption.	Low	Blocked
Verified Profile and Public Presence: View verified profile from dashboard	A signed-in user opens their own public-facing profile from the dashboard and reviews how verified achievements are presented. The profile should display evidence-backed cards with source links and confidence information.	High	Failed
Notifications and Settings: View notifications feed	A signed-in user opens notifications and reviews recent updates. The feed should display current notification items and recent activity without errors.	Low	Passed
Authentication and Account Access: Reset a forgotten password	A user can request a password reset from the recovery page. The flow should allow requesting a reset link and progressing to set a new password.	Medium	Failed
Proof Wallet and Source Connections: View connected sources	A user opens the sources area to review connected evidence. The page shows linked sources and their current status.	Medium	Passed
Marketing and Public Information: Visitor reads product documentation	A visitor can open a public information page and review product details. The page should display helpful documentation or product content for learning about Orin.	Low	Passed
Authentication and Account Access: Sign in to Orin	An existing user can log in and open the dashboard. Successful sign in should land the user in the authenticated experience.	High	Passed
Proof Wallet and Source Connections: Add a new source connection	A user connects a new evidence source from the source setup flow. The connected source is saved and appears in the sources area afterward.	High	Passed
Contact and Support: Require contact message details	A visitor must provide the required contact details before sending an inquiry. Submitting an incomplete form should keep the message from being sent and surface validation feedback.	Low	Passed
Proof Wallet and Source Connections: Manage account integrations	A user opens settings to review available integrations and adjust them. The integration settings are saved for the account.	Medium	Passed
Proof Cards: Share a proof card	A user creates a shareable record for a proof card. The test confirms sharing produces an accessible proof link.	Medium	Failed
AI Chat and Artifact Experience: Review tool evidence in AI chat	A user opens an AI chat response and inspects the supporting evidence used by the assistant. The test verifies that tool call details and citations are visible and can be reviewed from the response.	Medium	Blocked
Opportunity Matching and Search: Filter opportunities by category	A user narrows the opportunity list by internship or full-time category. The displayed results should update to match the selected filter.	Low	Passed
Skill Analysis and Skill Gap Engine: Check skill gaps for a target role	A user selects a target role and runs a comparison against their portfolio. The result should show missing skills and a gap analysis for that role.	High	Failed
Skill Analysis and Skill Gap Engine: Review extracted skills on Analytics	A user opens the analytics view and inspects the extracted skills summary. The page should show skill-related analysis and confidence information for the current portfolio.	High	Passed

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
University Dashboard: View university dashboard student outcomes	A career services user opens the university dashboard and reviews aggregated cohort data. The dashboard should display student progress and outcome metrics for the current view.	Medium	Passed
Notifications and Settings: Mark a notification as read	A signed-in user opens notifications and marks one item as read. The notification state should update so the item no longer appears unread.	Low	Blocked
Proof Cards: View proof card evidence	A user opens an existing proof card to inspect its evidence and confidence information. The test confirms the proof details page shows source-linked evidence and confidence information for the card.	High	Failed
Opportunity Matching and Search: Save a recommended opportunity	A user saves one of the recommended opportunities for later review. The saved state should persist after the action and be reflected in the browsing experience.	Medium	Passed
Proof Cards: Create a verified proof card	A user creates a new verified achievement card from their work. The test confirms the card is generated successfully and appears as a new proof record.	High	Failed
Subscription and Pricing: View current subscription status	A signed-in user opens the billing area and checks the current plan and usage limits. The page should clearly show the active subscription status and account tier information.	Low	Passed
Skill Analysis and Skill Gap Engine: Track skill progress on Analytics	A user opens analytics to monitor improvement over time. The page should display progress indicators and milestone information for skill development.	Medium	Passed
Notifications and Settings: Update notification preferences	A signed-in user changes how product updates are delivered from settings. The new notification preferences should save successfully and remain applied after submission.	Medium	Passed
Subscription and Pricing: Open the pricing page and compare plans	A visitor reviews the public pricing page before deciding whether to upgrade. The page should present the available tiers and let the user compare their features.	Low	Passed
Contact and Support: Send a contact inquiry	A visitor can open the support contact page, enter contact details and a message, and submit the inquiry successfully. The form should accept valid input and confirm that the message was sent.	Low	Passed
AI Career Coach and Learning Paths: AI coach generates a learning path for a target role	A user selects a target role and requests a learning plan. The coach should analyze the user's gaps and present a role-specific, week-by-week learning path.	High	Blocked
Opportunity Matching and Search: View opportunity details from a recommendation	A user opens a recommendation to inspect its details before deciding whether to save or pursue it. The detail view should present the opportunity information needed for review.	Low	Failed
Marketing and Public Information: Visitor explores the landing page	A visitor can open the homepage and understand what Orin offers. The page should present the main product overview and clear paths to get started or sign in.	Low	Passed
Proof Cards: Edit an existing proof card	A user updates the contents of an existing proof card. The test confirms the changes save successfully and the updated proof remains accessible.	Medium	Passed
Verified Profile and Public Presence: Update profile details	A signed-in user edits profile information and saves the changes successfully. The updated profile data should persist after saving.	Medium	Passed
AI Career Coach and Learning Paths: AI coach shows generated coaching notes	A user opens coaching notes and reviews the generated advice. The page should present clear notes and practical recommendations the user can act on.	Medium	Failed

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
AI Chat and Artifact Experience: Inspect chat attachments in conversation	A user adds an uploaded file to an AI chat and confirms it appears in the conversation. The test verifies that the attachment is shown as part of the chat context and remains available for the exchange.	Medium	Passed
AI Career Coach and Learning Paths: AI coach returns portfolio-based career guidance	A user opens the career coach and asks for guidance based on their portfolio and goals. The coach should respond with personalized advice and next-step recommendations relevant to the user's profile.	High	Passed
Verified Profile and Public Presence: View public profile as a recruiter	A recruiter or visitor opens a public profile page and reviews verified proof cards. The page should present source-linked achievements that are visible without signing in.	High	Failed
Proof Cards: View a public profile with verified proof cards	A recruiter or visitor opens a public profile and sees the user's proof-based presentation. The test confirms verified proof cards and source-linked achievements are visible on the public page.	High	Failed
Opportunity Matching and Search: Browse matched opportunities	A user opens the opportunities area and reviews recommendations ranked by fit. The page should show opportunity cards with match information and supporting skill gap context.	High	Passed
AI Chat and Artifact Experience: Send a streamed AI chat message	A user starts a conversation in the AI chat and receives a streamed reply. The test verifies that the conversation updates with an in-progress response and then completes successfully.	High	Passed
Opportunity Matching and Search: Track an opportunity application	A user updates the progress of an opportunity they are pursuing. The new application status should be saved and reflected in the tracking view.	Low	Blocked

3 Test Execution Breakdown

Orin-Three.Vercel.App Failed & Blocked Test Details

AI Chat and Artifact Experience: Stop a running AI chat generation

ATTRIBUTES	
Status	Blocked
Priority	Low
Description	A user cancels an in-progress chat generation before it finishes. The test verifies that generation stops and the conversation reflects the interruption.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714933775318//tmp/8c067a75-6670-42f2-b05d-55b454f3b26e/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the page header to open the sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link', name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> input
        # you@example.com email field
        elem = page.locator('[id="email"]')
        await elem.wait_for(state="visible", timeout=10000)
```

```
52          await elem.fill("test@email.com")
53
54          # -> input
55          # Enter your password password field
56          elem = page.locator('[id="password"]')
57          await elem.wait_for(state="visible", timeout=10000)
58          await elem.fill("Test@123")
59
60          # -> click
61          # Sign in button
62          elem = page.get_by_role('button', name='Sign in', exact=True)
63          await elem.click(timeout=10000)
64
65          # -> Open the AI Chat page by clicking the 'AI Chat' link in
            the left sidebar and wait for the chat UI to load.
66          # AI Chat link
67          elem = page.get_by_role('link', name='AI Chat', exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Fill the chat input with a long, multi-section prompt
            that will trigger a lengthy response, then click the Send
            button to start generation.
71          # Ask Orin anything... text area
72          elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
73          await elem.wait_for(state="visible", timeout=10000)
74          await elem.fill("Please write a very long, detailed,
            multi-section step-by-step guide (about 2500+ words) to build
            a production-ready full-stack web application with these
            requirements: authentication (email/password + social
            optional), file uploads with virus scanning, background job
            processing for long tasks, a REST API and a GraphQL layer,
            database schema design (Postgres), user roles and permissions,
            CI/CD pipeline to build and deploy to a cloud provider,
            monitoring and logging, performance and security best
            practices, and a canary deployment strategy. Include concrete
            code examples and commands using Node.js (Express or NestJS)
            for the backend and React for the frontend, migration examples
            (SQL), Dockerfile examples, GitHub Actions workflow for CI/CD,
            sample configuration for environment variables and secrets,
            TLS/HTTPS config, and step-by-step commands to set up the
            environment. Also include common troubleshooting steps and a
            summary checklist at the end. Provide clear code blocks and
            shell commands so the guide is reproducible.")
75
76          # -> Fill the chat input with a long, multi-section prompt
            that will trigger a lengthy response, then click the Send
            button to start generation.
77          # button
78          elem = page.locator('xpath=/html/body/div[2]/main/div/div/div
            [2]/div/div/div/div[3]/div[3]/div/div/button[2]')
79          await elem.click(timeout=10000)
80
81          # -> Type the long multi-section prompt into the 'Ask Orin
            anything...' chat textarea and click the 'Send' button to
            start response generation.
82          # Ask Orin anything... text area
83          elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
```

```
84            await elem.wait_for(state="visible", timeout=10000)
85            await elem.fill("Please write a very long, detailed,
              multi-section step-by-step guide (about 2500+ words) to build
              a production-ready full-stack web application with these
              requirements: authentication (email/password + social
              optional), file uploads with virus scanning, background job
              processing for long tasks, a REST API and a GraphQL layer,
              database schema design (Postgres), user roles and permissions,
              CI/CD pipeline to build and deploy to a cloud provider,
              monitoring and logging, performance and security best
              practices, and a canary deployment strategy. Include concrete
              code examples and commands using Node.js (Express or NestJS)
              for the backend and React for the frontend, migration examples
              (SQL), Dockerfile examples, GitHub Actions workflow for CI/CD,
              sample configuration for environment variables and secrets,
              TLS/HTTPS config, and step-by-step commands to set up the
              environment. Also include common troubleshooting steps and a
              summary checklist at the end. Provide clear code blocks and
              shell commands so the guide is reproducible.")
86
87            # -> Type the long multi-section prompt into the 'Ask Orin
              anything...' chat textarea and click the 'Send' button to
              start response generation.
88            # button
89            elem = page.locator('xpath=/html/body/div[2]/main/div/div/div
              [3]/div/div/button[2]')
90            await elem.click(timeout=10000)
91
92            # -> Click the visible 'Retry' button in the failed response
              bubble to re-start generation of the last message, then
              observe the UI for a Stop/Cancel control to interrupt the
              in-progress generation.
93            # Retry button
94            elem = page.locator('xpath=/html/body/div[2]/main/div/div/div
              [2]/div/div/div/div[2]/div/div/div[3]/button')
95            await elem.click(timeout=10000)
96
97            # -> Send the long prompt by pressing Enter in the 'Ask Orin
              anything...' textarea to start generation so the Stop/Cancel
              control appears.
98            # Ask Orin anything... text area
99            elem = page.get_by_placeholder('Ask Orin anything...',
              exact=True)
100           await elem.wait_for(state="visible", timeout=10000)
101           await elem.fill("Please write a very long, detailed,
              multi-section step-by-step guide (about 2500+ words) to build
              a production-ready full-stack web application with these
              requirements: authentication (email/password + social
              optional), file uploads with virus scanning, background job
              processing for long tasks, a REST API and a GraphQL layer,
              database schema design (Postgres), user roles and permissions,
              CI/CD pipeline to build and deploy to a cloud provider,
              monitoring and logging, performance and security best
              practices, and a canary deployment strategy. Include concrete
              code examples and commands using Node.js (Express or NestJS)
              for the backend and React for the frontend, migration examples
              (SQL), Dockerfile examples, GitHub Actions workflow for CI/CD,
              sample configuration for environment variables and secrets,
              TLS/HTTPS config, and step-by-step commands to set up the
```

```
            environment. Also include common troubleshooting steps and a
            summary checklist at the end. Provide clear code blocks and
            shell commands so the guide is reproducible.")
102
103         # -> Type the long prompt into the 'Ask Orin anything...'
            textarea and press Enter to send; if a response generation
            begins, click the 'Stop' or 'Cancel' control to interrupt it
            and verify the conversation shows the interruption.
104         # Ask Orin anything... text area
105         elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
106         await elem.wait_for(state="visible", timeout=10000)
107         await elem.fill("Please write a very long, detailed,
            multi-section step-by-step guide (about 2500+ words) to build
            a production-ready full-stack web application with these
            requirements: authentication (email/password + social
            optional), file uploads with virus scanning, background job
            processing for long tasks, a REST API and a GraphQL layer,
            database schema design (Postgres), user roles and permissions,
            CI/CD pipeline to build and deploy to a cloud provider,
            monitoring and logging, performance and security best
            practices, and a canary deployment strategy. Include concrete
            code examples and commands using Node.js (Express or NestJS)
            for the backend and React for the frontend, migration examples
            (SQL), Dockerfile examples, GitHub Actions workflow for CI/CD,
            sample configuration for environment variables and secrets,
            TLS/HTTPS config, and step-by-step commands to set up the
            environment. Also include common troubleshooting steps and a
            summary checklist at the end. Provide clear code blocks and
            shell commands so the guide is reproducible.")
108
109         # -> Type the long multi-section prompt into the 'Ask Orin
            anything...' textarea and press Enter to send; if response
            generation begins, click the 'Stop' or 'Cancel' control to
            interrupt it and verify the conversation shows the
            interruption.
110         # Ask Orin anything... text area
111         elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
112         await elem.wait_for(state="visible", timeout=10000)
113         await elem.fill("Please write a very long, detailed,
            multi-section step-by-step guide (about 2500+ words) to build
            a production-ready full-stack web application with these
            requirements: authentication (email/password + social
            optional), file uploads with virus scanning, background job
            processing for long tasks, a REST API and a GraphQL layer,
            database schema design (Postgres), user roles and permissions,
            CI/CD pipeline to build and deploy to a cloud provider,
            monitoring and logging, performance and security best
            practices, and a canary deployment strategy. Include concrete
            code examples and commands using Node.js (Express or NestJS)
            for the backend and React for the frontend, migration examples
            (SQL), Dockerfile examples, GitHub Actions workflow for CI/CD,
            sample configuration for environment variables and secrets,
            TLS/HTTPS config, and step-by-step commands to set up the
            environment. Also include common troubleshooting steps and a
            summary checklist at the end. Provide clear code blocks and
            shell commands so the guide is reproducible.")
114
```

```
            # --> Assertions to verify final state
            # Assert: Verify the response generation is interrupted
            assert False, "Expected: Verify the response generation is
            interrupted (could not be verified on the page)"

            # --> Test blocked by environment/access constraints during
            agent run
            # Reason: TEST BLOCKED The test could not be run — response
            generation cannot be started because the AI Chat interface
            returns persistent backend errors. Observations: - The AI Chat
            conversation displays 'Something went wrong. Please try again.
            ' for each send attempt and Retry, with no running generation
            visible. - Multiple send methods were used (Send button click,
            Enter key, Retry) and all returned the...
            raise AssertionError("Test blocked during agent run: " + "TEST
            BLOCKED The test could not be run \u2014 response generation
            cannot be started because the AI Chat interface returns
            persistent backend errors. Observations: - The AI Chat
            conversation displays 'Something went wrong. Please try again.
            ' for each send attempt and Retry, with no running generation
            visible. - Multiple send methods were used (Send button click,
            Enter key, Retry) and all returned the..." + " — the exported
            script cannot reproduce a PASS in this environment.")
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST BLOCKED The test could not be run — response generation cannot be started because the AI Chat interface returns persistent backend errors. Observations: - The AI Chat conversation displays 'Something went wrong. Please try again.' for each send attempt and Retry, with no running generation visible. - Multiple send methods were used (Send button click, Enter key, Retry) and all returned the same error; no Stop/Cancel control ever appeared. - The prompt remains in the input area and no partial output or in-progress generation is present.

Verified Profile and Public Presence: View verified profile from dashboard

ATTRIBUTES

Status	Failed
Priority	High
Description	A signed-in user opens their own public-facing profile from the dashboard and reviews how verified achievements are presented. The profile should display evidence-backed cards with source links and confidence information.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714416644576//tmp/7ef94d65-4b40-4e80-b5ec-2954ec73ffb3/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the site header to open the Sign in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link', name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with test@email.com, fill the 'Password' field with Test@123, then click the 'Sign in' button.
        # you@example.com email field
```

```
            elem = page.locator('[id="email"]')
            await elem.wait_for(state="visible", timeout=10000)
            await elem.fill("test@email.com")

            # -> Fill the 'Email address' field with test@email.com, fill the 'Password' field with Test@123, then click the 'Sign in' button.
            # Enter your password password field
            elem = page.locator('[id="password"]')
            await elem.wait_for(state="visible", timeout=10000)
            await elem.fill("Test@123")

            # -> Fill the 'Email address' field with test@email.com, fill the 'Password' field with Test@123, then click the 'Sign in' button.
            # Sign in button
            elem = page.get_by_role('button', name='Sign in', exact=True)
            await elem.click(timeout=10000)

            # -> Click the 'View Profile' button in the dashboard header to open the public profile page.
            # View Profile link
            elem = page.get_by_role('link', name='View Profile', exact=True)
            await elem.click(timeout=10000)

            # --> Assertions to verify final state

            # --> Verify the verified profile page is displayed
            # Assert: Expected profile to show at least 1 verified proof.
            await expect(page.locator("xpath=/html/body/main/main/section[1]/div[2]/div/div/div[1]/div[2]").nth(0)).to_contain_text("1 Verified Proofs", timeout=15000), "Expected profile to show at least 1 verified proof."
            # Assert: Expected profile to display confidence information for verified achievements.
            await expect(page.locator("xpath=/html/body/main/main/section[1]/div[2]/div/div/div[1]/div[2]/span").nth(0)).to_contain_text("Confidence", timeout=15000), "Expected profile to display confidence information for verified achievements."
            # Assert: Verify verified achievement cards and source evidence are shown
            assert False, "Expected: Verify verified achievement cards and source evidence are shown (could not be verified on the page)"
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())
```

Error

RESULT: The public profile page was opened and inspected. Observations: - The profile header shows username 'test' and the role 'Verified Exploration Expert'. - The main content displays the message 'No public proofs yet' and 'This profile is just getting started. Check back soon!', and there are 0 Verified Proofs, 0 Skills Proven, 0 Profile Views, 0 Connected Sources. - No evidence-backed achievement cards, source links, or confidence information are present on this public profile page. CONCLUSION: The feature (public-facing verified achievement cards with source links and confidence details) is not present for this profile. TEST BLOCKED/NOT AVAILABLE: The profile contains no public proofs to verify; therefore the requested verification (inspect evidence-backed proof cards and source links/confidence data) cannot be completed because the content is absent. If further action is desired, add or publish proofs in the dashboard (Add Proof / connect sources) and re-check the public profile.

Cause

The public profile is likely connected to an empty or incorrect production data store, or the hosting environment is serving a stale/incomplete build that does not include the verified proof records. In practice this can happen if the app is pointed at the wrong database/API in production, environment variables are misconfigured, public-proof queries are failing silently, or the site is cached/revalidated before newly published proofs are available. As a result, the profile renders only the default placeholder state ('No public proofs yet') instead of evidence-backed achievement cards with source and confidence details.

Fix

Verify the production deployment is serving the correct profile data source and that published proofs are being persisted and exposed to the public profile API/page. Check for environment/config issues on hosting (wrong database/project, missing env vars, failed build/deploy, stale cache, or SSR/ISR not revalidating). If the dashboard is saving proofs elsewhere or not marking them public, fix the publish workflow so proofs, source links, and confidence metadata are written to the production backend and rendered on the public profile after deployment. Then clear caches/redeploy and confirm the public profile endpoint returns the expected proof cards for test accounts.

Authentication and Account Access: Reset a forgotten password

ATTRIBUTES

Status	Failed
Priority	Medium
Description	A user can request a password reset from the recovery page. The flow should allow requesting a reset link and progressing to set a new password.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714449541918//tmp/248bb8a2-eda5-4cbc-9ce1-b21d8424938f/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the 'Reset your password' page by navigating to the site's Reset Password page so the email field can be filled.
        await page.goto("https://orin-three.vercel.app/reset-password")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Fill the 'Email address' field with test@email.com and click the 'Send reset link' button.
        # you@example.com email field
```

```
            elem = page.locator('[id="email"]')
            await elem.wait_for(state="visible", timeout=10000)
            await elem.fill("test@email.com")

            # -> Fill the 'Email address' field with test@email.com and
            click the 'Send reset link' button.
            # Send reset link button
            elem = page.get_by_role('button', name='Send reset link',
            exact=True)
            await elem.click(timeout=10000)

            # --> Assertions to verify final state

            # --> Verify a password reset confirmation is visible
            # Assert: Expected a password reset confirmation saying 'We've
            sent a password reset link' to be visible.
            await expect(page.locator("xpath=/html/body/section").nth(0)).
            to_contain_text("We've sent a password reset link",
            timeout=15000), "Expected a password reset confirmation saying
            'We've sent a password reset link' to be visible."
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST FAILURE The password reset request could not be completed because the reset form rejected the provided email address and blocked submission. Observations: - The page displays the validation message: Email address "test@email.com" is invalid. - The Email address field contains the value "test@email.com" and the "Send reset link" button is present. - Submission was attempted but no confirmation or next-step UI to set a new password appeared.

Cause

The hosting-side failure is likely caused by a deployment/configuration mismatch in the password reset flow: the production build is rejecting otherwise valid email addresses due to incorrect validation logic or a misconfigured reset endpoint. This could be from a strict email pattern, stale frontend code, backend validation rules, or an environment variable/API route issue causing the form to treat test@email.com as invalid and prevent the reset request from being submitted.

Fix

Check the hosted password-reset form validation/schema and the email input handling. The app may be using an overly strict validator on the server or in the deployed frontend (for example, a broken regex, a schema expecting a different format, or a double-encoded/trimmed value) that incorrectly rejects valid emails like test@email.com. Update the deployed build so the reset request accepts standard RFC-compliant email addresses, ensure the input is sent exactly as entered, and verify the reset endpoint responds with the next-step/reset confirmation flow instead of blocking submission.

Proof Cards: Share a proof card

ATTRIBUTES

Status	Failed
Priority	Medium
Description	A user creates a shareable record for a proof card. The test confirms sharing produces an accessible proof link.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714466522565//tmp/5b76d4ec-c6e3-4c29-a414-e9a53e3c46b5/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the top navigation to open the
        sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the Email address field with 'test@email.com', fill
        the Password field with 'Test@123', then click the 'Sign in'
        button to submit the login form.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the Email address field with 'test@email.com', fill
              the Password field with 'Test@123', then click the 'Sign in'
              button to submit the login form.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the Email address field with 'test@email.com', fill
              the Password field with 'Test@123', then click the 'Sign in'
              button to submit the login form.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Open the proof's details page by clicking the 'View
              Details' link on a proof card (the card titled
              'explore-test-123: Updated React Component').
66            # View Details link
67            elem = page.get_by_text('Testing', exact=True).locator
              ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
              name='View Details', exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Click the 'Share' button on the proof details page (the
              top-right 'Share' button) to request sharing of the proof.
71            # Share button
72            elem = page.get_by_role('button', name='Share', exact=True)
73            await elem.click(timeout=10000)
74
75            # --> Assertions to verify final state
76
77            # --> Verify a shareable proof link is created
78            # Assert: Expected URL to contain '/dashboard/proof/
              933af144-3770-439f-97d3-8f00979efc1c' so the proof details
              page remained open after sharing.
79            await expect(page).to_have_url(re.compile("/dashboard/proof/
              933af144\\-3770\\-439f\\-97d3\\-8f00979efc1c"),
              timeout=15000), "Expected URL to contain '/dashboard/proof/
              933af144-3770-439f-97d3-8f00979efc1c' so the proof details
              page remained open after sharing."
80            await page.locator("xpath=/html/body/div[2]/main/div/article/
              header/div[2]/form/button").nth(0).scroll_into_view_if_needed()
81            # Assert: Expected the 'Share' button to be visible on the
              proof details page after requesting a share.
82            await expect(page.locator("xpath=/html/body/div[2]/main/div/
              article/header/div[2]/form/button").nth(0)).to_be_visible
              (timeout=15000), "Expected the 'Share' button to be visible on
              the proof details page after requesting a share."
83            await asyncio.sleep(5)
84
85        finally:
86            if context:
87                await context.close()
88            if browser:
```

```
            await browser.close()
        if pw:
            await pw.stop()

asyncio.run(run_test())

```

Error

TEST FAILURE Creating a shareable proof link failed — the share API returned an error instead of producing a link. Observations: - After clicking the 'Share' button, the browser opened /api/proof/share and the page displayed the JSON error: {"error":"Invalid request body"}. - No shareable link, share confirmation, or success message was visible on the proof details page after the action.

Cause

The share endpoint /api/proof/share is receiving a request body that does not match what the backend expects, causing it to return {"error":"Invalid request body"}. This is likely due to a frontend/backend contract mismatch introduced in the deployed hosting version: the Share button may be posting an empty or malformed body, using the wrong HTTP method, or omitting a required proof identifier. As a result, the API cannot generate the proof link and the share flow fails.

Fix

Inspect the client-side Share action and the /api/proof/share handler contract. The most likely issue is that the frontend is sending the wrong payload shape or content type (for example, missing required proofId/proofSlug/share fields, sending a GET request instead of POST, or not setting application/json), so the server rejects it as an invalid body. Update the client to send the exact body the endpoint expects, and make the API more tolerant by validating inputs clearly and returning a helpful 4xx response. Also verify the hosting deployment has the latest serverless function code and environment variables, since an outdated or misconfigured API route can produce the same invalid-body error.

AI Chat and Artifact Experience: Review tool evidence in AI chat

ATTRIBUTES

Status	Blocked
Priority	Medium
Description	A user opens an AI chat response and inspects the supporting evidence used by the assistant. The test verifies that tool call details and citations are visible and can be reviewed from the response.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714669804175//tmp/350f3e1c-7bad-4aab-831f-30cc800bbbc9/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the top navigation to open the
        sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Enter the email into the Email address field, enter the
        password into the Password field, and click the 'Sign in'
        button to sign in.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Enter the email into the Email address field, enter the
              password into the Password field, and click the 'Sign in'
              button to sign in.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Enter the email into the Email address field, enter the
              password into the Password field, and click the 'Sign in'
              button to sign in.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the 'AI Chat' link in the left navigation to open
              the AI Chat page.
66            # AI Chat link
67            elem = page.get_by_role('link', name='AI Chat', exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Click the 'Analyze my portfolio' button to start a
              portfolio analysis conversation and submit the request.
71            # 📊 Analyze my portfolio button
72            elem = page.get_by_role('button', name='📊 Analyze my
              portfolio', exact=True)
73            await elem.click(timeout=10000)
74
75            # -> Click the chat's orange send button to submit the
              'Analyze my portfolio' request, then wait for the assistant's
              response to finish streaming.
76            # button
77            elem = page.locator('xpath=/html/body/div[2]/main/div/div/div
              [2]/div/div/div/div[3]/div[3]/div/div/button[2]')
78            await elem.click(timeout=10000)
79
80            # -> Click the 'Retry' button in the error message to retry
              the assistant response, then wait for the assistant to finish
              streaming a successful reply.
81            # Retry button
82            elem = page.get_by_role('button', name='Retry', exact=True)
83            await elem.click(timeout=10000)
84
85            # -> Click the visible 'Retry' button in the chat error
              message and wait for the assistant's response to stream so the
              reply can be inspected for tool call details and citations.
86            # Retry button
87            elem = page.get_by_role('button', name='Retry', exact=True)
88            await elem.click(timeout=10000)
89
90            # --> Assertions to verify final state
91            # Assert: Verify tool call details are displayed
92            assert False, "Expected: Verify tool call details are
              displayed (could not be verified on the page)"
93            # Assert: Verify citations are displayed
```

```
94            assert False, "Expected: Verify citations are displayed (could
              not be verified on the page)"
95
96            # --> Test blocked by environment/access constraints during
              agent run
97            # Reason: TEST BLOCKED The test could not be run — the
              assistant response could not be obtained because an error
              state persists in the AI Chat UI. Observations: - The chat
              shows a persistent error banner: 'Something went wrong. Please
              try again.' - Clicking the 'Retry' button did not resolve the
              error; no assistant reply appeared after multiple attempts. -
              The AI response needed to verify tool call detai...
98            raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The test could not be run \u2014 the assistant
              response could not be obtained because an error state persists
              in the AI Chat UI. Observations: - The chat shows a persistent
              error banner: 'Something went wrong. Please try again.' -
              Clicking the 'Retry' button did not resolve the error; no
              assistant reply appeared after multiple attempts. - The AI
              response needed to verify tool call detai..." + " — the
              exported script cannot reproduce a PASS in this environment.")
99            await asyncio.sleep(5)
100
101       finally:
102           if context:
103               await context.close()
104           if browser:
105               await browser.close()
106           if pw:
107               await pw.stop()
108
109   asyncio.run(run_test())
110
```

Error

TEST BLOCKED The test could not be run — the assistant response could not be obtained because an error state persists in the AI Chat UI. Observations: - The chat shows a persistent error banner: 'Something went wrong. Please try again.' - Clicking the 'Retry' button did not resolve the error; no assistant reply appeared after multiple attempts. - The AI response needed to verify tool call details and citations could not be retrieved.

Skill Analysis and Skill Gap Engine: Check skill gaps for a target role

ATTRIBUTES

Status	Failed
Priority	High
Description	A user selects a target role and runs a comparison against their portfolio. The result should show missing skills and a gap analysis for that role.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714484926294//tmp/24ee86da-a684-4957-b4ff-153989a6c1fb/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the page header to open the
        sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with 'test@email.com',
        fill the 'Password' field with 'Test@123', then click the
        'Sign in' button.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the 'Email address' field with 'test@email.com',
              fill the 'Password' field with 'Test@123', then click the
              'Sign in' button.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the 'Email address' field with 'test@email.com',
              fill the 'Password' field with 'Test@123', then click the
              'Sign in' button.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the 'Skill Gaps' link in the left sidebar to open
              the Skill Gap Engine page.
66            # Skill Gaps link
67            elem = page.get_by_role('link', name='Skill Gaps', exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Select the 'Data Scientist' target role and then click
              the 'Analyze Skill Gaps' button to run the gap analysis and
              reveal missing skills and gap results.
71            # Data Scientist button
72            elem = page.get_by_role('button', name='Data Scientist',
              exact=True)
73            await elem.click(timeout=10000)
74
75            # -> Select the 'Data Scientist' target role and then click
              the 'Analyze Skill Gaps' button to run the gap analysis and
              reveal missing skills and gap results.
76            # Analyze Skill Gaps button
77            elem = page.get_by_role('button', name='Analyze Skill Gaps',
              exact=True)
78            await elem.click(timeout=10000)
79
80            # --> Assertions to verify final state
81            # Assert: Verify missing skills are displayed
82            assert False, "Expected: Verify missing skills are displayed
              (could not be verified on the page)"
83            # Assert: Verify gap analysis results are displayed
84            assert False, "Expected: Verify gap analysis results are
              displayed (could not be verified on the page)"
85            await asyncio.sleep(5)
86
87        finally:
88            if context:
89                await context.close()
90            if browser:
91                await browser.close()
92            if pw:
93                await pw.stop()
94
95    asyncio.run(run_test())
```

96

Error

TEST FAILURE The skill gap analysis did not produce the expected results — an error was shown instead of missing skills and a gap analysis. Observations: - The page displays the error message: "Unexpected token '<', "<!DOCTYPE "... is not valid JSON" in red beneath the Analyze area. - No missing skills list or gap analysis results are visible after selecting 'Data Scientist' and clicking 'Analyze Skill Gaps'. - The Data Scientist role was selected and the Analyze action was performed before the error appeared.

Cause

The error "Unexpected token '<', \"<!DOCTYPE \"... is not valid JSON" indicates the frontend tried to parse an HTML document as JSON. On the hosting side this usually means the Analyze Skill Gaps request is being routed to the site's HTML fallback (or a 404 page) instead of the intended JSON API response. Common causes are a missing/misdeployed API route, incorrect Vercel rewrite/proxy rules, or an unset/wrong environment variable pointing the client to the API.

Fix

Check the Analyze Skill Gaps frontend request target and the hosting rewrites/proxy configuration. The client is likely calling an API endpoint that is returning the app's HTML fallback page (index.html/404) instead of JSON, which suggests the backend route is missing, misrouted, or the environment variable for the API base URL is incorrect on Vercel. Fix by ensuring the skill-gap endpoint exists and returns JSON, and configure Vercel rewrites so API requests go to the correct serverless function or external backend rather than the SPA route. Also verify the deployed function/runtime is healthy and that the production API URL used by the frontend is set correctly.

Notifications and Settings: Mark a notification as read

ATTRIBUTES

Status	Blocked
Priority	Low
Description	A signed-in user opens notifications and marks one item as read. The notification state should update so the item no longer appears unread.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714400744101//tmp/080018a3-aac4-4b93-9811-a7851fb2fc35/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> navigate
        await page.goto("https://orin-three.vercel.app/signin")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Fill the email field with test@email.com, fill the password field with Test@123, then click the 'Sign in' button to submit the form.
        # you@example.com email field
```

```
52            elem = page.locator('[id="email"]')
53            await elem.wait_for(state="visible", timeout=10000)
54            await elem.fill("test@email.com")
55
56            # -> Fill the email field with test@email.com, fill the
              password field with Test@123, then click the 'Sign in' button
              to submit the form.
57            # Enter your password password field
58            elem = page.locator('[id="password"]')
59            await elem.wait_for(state="visible", timeout=10000)
60            await elem.fill("Test@123")
61
62            # -> Fill the email field with test@email.com, fill the
              password field with Test@123, then click the 'Sign in' button
              to submit the form.
63            # Sign in button
64            elem = page.get_by_role('button', name='Sign in', exact=True)
65            await elem.click(timeout=10000)
66
67            # -> Click the 'Notifications' button in the left sidebar to
              open the notifications panel so a notification can be marked
              as read.
68            # 0 Notifications button
69            elem = page.get_by_role('button', name='0 Notifications',
              exact=True)
70            await elem.click(timeout=10000)
71
72            # --> Assertions to verify final state
73            # Assert: Verify the notification state updates
74            assert False, "Expected: Verify the notification state updates
              (could not be verified on the page)"
75
76            # --> Test blocked by environment/access constraints during
              agent run
77            # Reason: TEST BLOCKED The test could not be run — there are
              no notifications available to mark as read. Observations: -
              The Notifications panel is open and shows the message 'No
              notifications yet'. - No notification items or unread
              indicators are visible in the panel.
78            raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The test could not be run \u2014 there are no
              notifications available to mark as read. Observations: - The
              Notifications panel is open and shows the message 'No
              notifications yet'. - No notification items or unread
              indicators are visible in the panel." + " — the exported
              script cannot reproduce a PASS in this environment.")
79            await asyncio.sleep(5)
80
81        finally:
82            if context:
83                await context.close()
84            if browser:
85                await browser.close()
86            if pw:
87                await pw.stop()
88
89    asyncio.run(run_test())
90
```

Error

TEST BLOCKED The test could not be run — there are no notifications available to mark as read. Observations: - The Notifications panel is open and shows the message 'No notifications yet'. - No notification items or unread indicators are visible in the panel.

Proof Cards: View proof card evidence

ATTRIBUTES

Status	Failed
Priority	High
Description	A user opens an existing proof card to inspect its evidence and confidence information. The test confirms the proof details page shows source-linked evidence and confidence information for the card.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714786372383//tmp/ffad080e-e480-43b4-8e73-4c3b049e70dd/result.webm

</> Test Code

```python
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the page header to open the sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link', name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with test@email.com and the 'Password' field with Test@123, then click the 'Sign in' button to submit the form.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the 'Email address' field with test@email.com and
              the 'Password' field with Test@123, then click the 'Sign in'
              button to submit the form.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the 'Email address' field with test@email.com and
              the 'Password' field with Test@123, then click the 'Sign in'
              button to submit the form.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the 'View Details' link on a proof card to open the
              proof details page and inspect evidence and confidence
              information.
66            # View Details link
67            elem = page.get_by_text('React Frontend
              ProjectProjectPending', exact=True).locator
              ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
              name='View Details', exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Scroll down the 'React Frontend Project' proof details
              page to reveal additional sections, then search the page for
              the labels 'evidence' (or 'Sources') and 'confidence' (or
              similar verification/confidence text).
71            await page.mouse.wheel(0, 300)
72
73            # -> Scroll the proof details page to the bottom to reveal any
              'Sources' or 'Evidence' section so it can be verified (if
              absent, report that source-linked evidence is not displayed).
74            await page.mouse.wheel(0, 300)
75
76            # --> Assertions to verify final state
77            # Assert: Verify source-linked evidence is displayed
78            assert False, "Expected: Verify source-linked evidence is
              displayed (could not be verified on the page)"
79            # Assert: Verify confidence information is displayed
80            assert False, "Expected: Verify confidence information is
              displayed (could not be verified on the page)"
81            await asyncio.sleep(5)
82
83        finally:
84            if context:
85                await context.close()
86            if browser:
87                await browser.close()
88            if pw:
89                await pw.stop()
90
91    asyncio.run(run_test())
92
```

Error

TEST FAILURE Source-linked evidence is not displayed on the proof details page for the opened proof card. Observations: - The proof details page shows a 'Verified' label with value 'No'. - No 'Sources' or 'Evidence' section or source links were found on the proof details page after multiple scrolls and searches.

Cause

The hosted app is likely serving an incomplete or incorrect production build/data contract for the proof details page. The most probable issue is that the proof detail view is not receiving or rendering the source/evidence metadata from the backend (for example due to a missing API response field, wrong environment variable, stale deployment, or a frontend condition that hides evidence whenever the proof is marked Verified: No). As a result, the page loads the proof but omits the Sources/Evidence section and source links.

Fix

Ensure the proof details page renders source-linked evidence for cards marked as verified or with available sources. On the hosting/app side, this likely means the deployed frontend is using a build or data source where the proof detail component either never receives the evidence payload, filters it out when `verified=false`, or fails due to an API/schema mismatch. Rebuild/redeploy with the correct environment/config, verify the proof details API returns `sources`/`evidence` fields, and update the UI to always show the Sources/Evidence section when evidence exists regardless of the Verified flag.

Proof Cards: Create a verified proof card

ATTRIBUTES

Status	Failed
Priority	High
Description	A user creates a new verified achievement card from their work. The test confirms the card is generated successfully and appears as a new proof record.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714713089566//tmp/9b74f768-76be-459c-b041-8243a23a195b/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link to open the sign-in page so
        credentials can be entered.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with test@email.com, fill
        the 'Password' field with Test@123, then click the 'Sign in'
        button to submit the login form.
        # you@example.com email field
```

```
50          elem = page.locator('[id="email"]')
51          await elem.wait_for(state="visible", timeout=10000)
52          await elem.fill("test@email.com")
53
54          # -> Fill the 'Email address' field with test@email.com, fill
            the 'Password' field with Test@123, then click the 'Sign in'
            button to submit the login form.
55          # Enter your password password field
56          elem = page.locator('[id="password"]')
57          await elem.wait_for(state="visible", timeout=10000)
58          await elem.fill("Test@123")
59
60          # -> Fill the 'Email address' field with test@email.com, fill
            the 'Password' field with Test@123, then click the 'Sign in'
            button to submit the login form.
61          # Sign in button
62          elem = page.get_by_role('button', name='Sign in', exact=True)
63          await elem.click(timeout=10000)
64
65          # -> Click the 'Add Proofs' link in the left sidebar to open
            the Create Proof (Add Proof) page.
66          # Add Proofs link
67          elem = page.get_by_role('link', name='Add Proofs', exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Paste a GitHub repository URL into the 'Paste a GitHub
            repo URL' field and click the 'Generate Proof' button to
            import the repo and generate a proof card.
71          # https://github.com/owner/repo url field
72          elem = page.get_by_placeholder('https://github.com/owner/
            repo', exact=True)
73          await elem.wait_for(state="visible", timeout=10000)
74          await elem.fill("https://github.com/orin-dev/orin")
75
76          # -> Paste a GitHub repository URL into the 'Paste a GitHub
            repo URL' field and click the 'Generate Proof' button to
            import the repo and generate a proof card.
77          # Generate Proof button
78          elem = page.get_by_role('button', name='Generate Proof',
            exact=True)
79          await elem.click(timeout=10000)
80
81          # -> Click the 'Generate Proof' button to retry importing the
            GitHub repository and observe whether the app proceeds to
            create a proof card or shows an error message.
82          # Generate Proof button
83          elem = page.get_by_role('button', name='Generate Proof',
            exact=True)
84          await elem.click(timeout=10000)
85
86          # -> Click the 'Next' button to proceed to the manual proof
            creation flow and fill in proof details.
87          # Next button
88          elem = page.get_by_role('button', name='Next', exact=True)
89          await elem.click(timeout=10000)
90
91          # -> Fill the 'Title' with 'Open Source Contributions to
            ORIN', fill 'Source URL' with 'https://github.com/orin-dev/
            orin', add a short Description, then click the 'Next' button
```

```
            to proceed with manual proof creation.
 92         # e.g. Open Source React Component Library text field
 93         elem = page.locator('[id="proofTitle"]')
 94         await elem.wait_for(state="visible", timeout=10000)
 95         await elem.fill("Open Source Contributions to ORIN")
 96
 97         # -> Fill the 'Title' with 'Open Source Contributions to
            ORIN', fill 'Source URL' with 'https://github.com/orin-dev/
            orin', add a short Description, then click the 'Next' button
            to proceed with manual proof creation.
 98         # e.g. https://github.com/user/repo url field
 99         elem = page.locator('[id="proofUrl"]')
100         await elem.wait_for(state="visible", timeout=10000)
101         await elem.fill("https://github.com/orin-dev/orin")
102
103         # -> Fill the 'Title' with 'Open Source Contributions to
            ORIN', fill 'Source URL' with 'https://github.com/orin-dev/
            orin', add a short Description, then click the 'Next' button
            to proceed with manual proof creation.
104         # Briefly describe what this proof demonstrates... text area
105         elem = page.locator('[id="proofDesc"]')
106         await elem.wait_for(state="visible", timeout=10000)
107         await elem.fill("Implemented features and documentation
            improvements to the ORIN repository to demonstrate
            contributions and project ownership.")
108
109         # -> Fill the 'Title' with 'Open Source Contributions to
            ORIN', fill 'Source URL' with 'https://github.com/orin-dev/
            orin', add a short Description, then click the 'Next' button
            to proceed with manual proof creation.
110         # Next button
111         elem = page.get_by_role('button', name='Next', exact=True)
112         await elem.click(timeout=10000)
113
114         # -> Click the 'Next' button on the manual proof creation form
            to proceed toward submitting the new proof and then verify
            that the new proof card appears on the dashboard or proofs
            list.
115         # Next button
116         elem = page.get_by_role('button', name='Next', exact=True)
117         await elem.click(timeout=10000)
118
119         # -> Click the 'Submit Proof Card' button to submit the new
            proof and then verify that the new proof card appears on the
            dashboard or proofs list.
120         # Submit Proof Card button
121         elem = page.get_by_role('button', name='Submit Proof Card',
            exact=True)
122         await elem.click(timeout=10000)
123
124         # --> Assertions to verify final state
125
126         # --> Verify a new proof card is displayed
127         # Assert: Expected dashboard verified count to show 1 verified.
128         await expect(page.locator("xpath=/html/body/div[2]/main/div/
            div/div/section/div[1]/div[1]/div[3]/p").nth(0)).to_have_text
            ("1 verified", timeout=15000), "Expected dashboard verified
            count to show 1 verified."
129         await asyncio.sleep(5)
```

```
130
131         finally:
132             if context:
133                 await context.close()
134             if browser:
135                 await browser.close()
136             if pw:
137                 await pw.stop()
138
139     asyncio.run(run_test())
140
```

Error

Summary of actions and verification: User request: Create a new verified achievement card from work and confirm the new proof record appears. What was done: - Signed in with test@email.com / Test@123. - Attempted to import GitHub repo 'https://github.com/orin-dev/orin' twice; both attempts failed with 'Failed to fetch GitHub repo'. - Used manual proof creation: Title='Open Source Contributions to ORIN', Source URL='https://github.com/orin-dev/orin', Description='Implemented features and documentation improvements to the ORIN repository to demonstrate contributions and project ownership.' - Submitted the proof via 'Submit Proof Card'. Observations (verifiable from the dashboard page and screenshot): - A new proof card titled 'Open Source Contributions to ORIN' appears in 'Your proof feed'. - The card shows a 'GitHub' source badge and an orange 'Pending' badge (status = Pending). - Recent Activity shows 'Created: Open Source Contributions to ORIN' with timestamp 'just now'. Conclusion vs. requested goal: - The proof record was generated successfully and is visible on the dashboard (requirement: 'card appears' — satisfied). - The proof is not shown as 'Verified' but as 'Pending' (requirement: 'verified achievement card' — NOT satisfied). Result: Partial success. The proof card creation and appearance are confirmed, but the verification status was not achieved in this run. If the test strictly requires the card to be marked 'Verified', the test did not complete successfully. Recommended next steps (optional): - If verification is expected to be automatic, investigate why the GitHub import failed earlier and whether verification depends on import success. - If verification requires a separate manual/administrative step, perform or simulate that step and re-check the card status. URLs/context: Current dashboard URL: https://orin-three.vercel.app/dashboard Recorded facts: Title: 'Open Source Contributions to ORIN'; Source URL used: 'https://github.com/orin-dev/orin'; Status observed: 'Pending'.

Cause

The hosting-side service appears unable to fetch the GitHub repository during the import/verification step, causing the proof workflow to stop at Pending instead of progressing to Verified. This could be due to a broken server-side fetch implementation, blocked outbound network access from the deployed environment, missing/invalid GitHub API credentials, rate limiting, CORS/redirect handling issues, or a backend logic bug where imported proofs are created but never transitioned to Verified after validation.

Fix

Investigate the GitHub repo fetch/import endpoint used by the dashboard: confirm outbound requests from the hosting environment are allowed, the GitHub URL parsing/normalization is correct, and the backend has valid credentials/headers for GitHub’s API or raw content access. Check server logs for the failed fetch attempts, rate limiting, CORS/serverless timeout issues, and any environment-variable misconfiguration (API token, allowlist, redirect handling). If verification depends on imported repo metadata, ensure the import pipeline persists the proof as Verified after successful validation, or add a separate verification job/endpoint that updates the status from Pending to Verified once the repo evidence is confirmed.

AI Career Coach and Learning Paths: AI coach generates a learning path for a target role

ATTRIBUTES

Status	Blocked
Priority	High
Description	A user selects a target role and requests a learning plan. The coach should analyze the user's gaps and present a role-specific, week-by-week learning path.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/178171474723801//tmp/5e75b63d-7d79-479a-9750-f2a5daba5ec5/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the page header to open the Sign in page so the email and password fields can be filled.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link', name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill 'test@email.com' into the Email address field, fill 'Test@123' into the Password field, then click the 'Sign in' button.
        # you@example.com email field
```

```
50          elem = page.locator('[id="email"]')
51          await elem.wait_for(state="visible", timeout=10000)
52          await elem.fill("test@email.com")
53
54          # -> Fill 'test@email.com' into the Email address field, fill
            'Test@123' into the Password field, then click the 'Sign in'
            button.
55          # Enter your password password field
56          elem = page.locator('[id="password"]')
57          await elem.wait_for(state="visible", timeout=10000)
58          await elem.fill("Test@123")
59
60          # -> Fill 'test@email.com' into the Email address field, fill
            'Test@123' into the Password field, then click the 'Sign in'
            button.
61          # Sign in button
62          elem = page.get_by_role('button', name='Sign in', exact=True)
63          await elem.click(timeout=10000)
64
65          # -> Click the 'AI Chat' navigation item in the left sidebar
            to open the AI coach interface.
66          # AI Chat link
67          elem = page.get_by_role('link', name='AI Chat', exact=True)
68          await elem.click(timeout=10000)
69
70          # -> Click the 'Analyze my portfolio' button in the AI Chat
            interface to start portfolio analysis and reveal role
            selection/options.
71          # 📊 Analyze my portfolio button
72          elem = page.get_by_role('button', name='📊 Analyze my
            portfolio', exact=True)
73          await elem.click(timeout=10000)
74
75          # -> Type a chat message requesting portfolio analysis for the
            target role 'Machine Learning Engineer' and ask for a
            role-specific 12-week, week-by-week learning plan (goals,
            resources, milestones), then send the message.
76          # Analyze my portfolio and suggest improvements for... text
            area
77          elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
78          await elem.wait_for(state="visible", timeout=10000)
79          await elem.fill("Analyze my portfolio for the target role
            'Machine Learning Engineer' and provide a role-specific
            12-week, week-by-week learning plan that addresses my skill
            gaps. For each week include: learning goals, suggested
            resources (articles, courses, projects), key milestones, and
            how to validate the skill (deliverable or assessment).")
80
81          # -> Click the 'Retry' button displayed with the error message
            to re-run the portfolio analysis request and wait for the AI
            response (look for skill gaps + a 12-week, week-by-week
            learning plan).
82          # Retry button
83          elem = page.get_by_role('button', name='Retry', exact=True)
84          await elem.click(timeout=10000)
85
86          # -> Click the 'Retry' button shown under the error message to
            re-run the portfolio analysis request and wait for the AI
```

```
            response (look for skill gaps and a 12-week, week-by-week
            learning plan).
 87         # Retry button
 88         elem = page.get_by_role('button', name='Retry', exact=True)
 89         await elem.click(timeout=10000)
 90
 91         # -> Click the 'Analyze my portfolio' quick-action button in
            the chat to re-run the portfolio analysis and wait for the AI
            response (look for skill gaps + a 12-week learning plan).
 92         # 📊 Analyze my portfolio button
 93         elem = page.get_by_role('button', name='📊 Analyze my
            portfolio', exact=True)
 94         await elem.click(timeout=10000)
 95
 96         # -> Start a new conversation, then submit the portfolio
            analysis request for the 'Machine Learning Engineer' role by
            typing the request into the chat input and pressing Enter to
            send.
 97         # Start a new conversation, then submit the portfolio analysis
            request for the 'Machine Learning Engineer' role by typing the
            request into the chat input and pressing Enter to send.
 98         elem = page.locator('xpath=/html/body/div[2]/main/div/div/
            header/div/div[2]/div')
 99         await elem.click(timeout=10000)
100
101         # -> Start a new conversation, then submit the portfolio
            analysis request for the 'Machine Learning Engineer' role by
            typing the request into the chat input and pressing Enter to
            send.
102         # Analyze my portfolio and suggest improvements for... text
            area
103         elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
104         await elem.wait_for(state="visible", timeout=10000)
105         await elem.fill("Analyze my portfolio for the target role
            'Machine Learning Engineer' and provide a role-specific
            12-week, week-by-week learning plan that addresses my skill
            gaps. For each week include: learning goals, suggested
            resources (articles, courses, projects), key milestones, and
            how to validate the skill (deliverable or assessment).")
106
107         # --> Assertions to verify final state
108         # Assert: Verify skill gaps are analyzed
109         assert False, "Expected: Verify skill gaps are analyzed (could
            not be verified on the page)"
110         # Assert: Verify a week-by-week learning path is displayed
111         assert False, "Expected: Verify a week-by-week learning path
            is displayed (could not be verified on the page)"
112
113         # --> Test blocked by environment/access constraints during
            agent run
114         # Reason: TEST BLOCKED The portfolio analysis feature could
            not be completed — the AI chat repeatedly returned an error
            and no analysis was produced. Observations: - The AI chat
            displays the message 'Something went wrong. Please try again.'
            - Clicking the 'Retry' button, using the 'Analyze my
            portfolio' quick-action, and re-sending the request multiple
            times did not produce a response. - No skill gap ana...
115         raise AssertionError("Test blocked during agent run: " + "TEST
```

```
            BLOCKED The portfolio analysis feature could not be completed
            \u2014 the AI chat repeatedly returned an error and no
            analysis was produced. Observations: - The AI chat displays
            the message 'Something went wrong. Please try again.' -
            Clicking the 'Retry' button, using the 'Analyze my portfolio'
            quick-action, and re-sending the request multiple times did
            not produce a response. - No skill gap ana..." + " — the
            exported script cannot reproduce a PASS in this environment.")
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST BLOCKED The portfolio analysis feature could not be completed — the AI chat repeatedly returned an error and no analysis was produced. Observations: - The AI chat displays the message 'Something went wrong. Please try again.' - Clicking the 'Retry' button, using the 'Analyze my portfolio' quick-action, and re-sending the request multiple times did not produce a response. - No skill gap analysis or 12-week, week-by-week learning plan was returned.

Opportunity Matching and Search: View opportunity details from a recommendation

ATTRIBUTES

Status	Failed
Priority	Low
Description	A user opens a recommendation to inspect its details before deciding whether to save or pursue it. The detail view should present the opportunity information needed for review.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714632388442//tmp/e59256b0-cf40-4b4f-910c-5eb4ed0cd8b9/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the header to open the sign-in
        page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with the test email, fill
        the 'Password' field with the provided password, then click
        the 'Sign in' button to authenticate.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the 'Email address' field with the test email, fill
              the 'Password' field with the provided password, then click
              the 'Sign in' button to authenticate.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the 'Email address' field with the test email, fill
              the 'Password' field with the provided password, then click
              the 'Sign in' button to authenticate.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the 'Opportunities' link in the left sidebar to
              open the Opportunities page so recommended listings can be
              inspected.
66            # Opportunities link
67            elem = page.get_by_role('link', name='Opportunities',
              exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Click the 'Apply' button on the 'Startup Mentorship
              Track' opportunity card to open its detail view for inspection.
71            # Apply link
72            elem = page.get_by_text('Y', exact=True).locator
              ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
              name='Apply', exact=True)
73            await elem.click(timeout=10000)
74
75            # -> Switch to the 'Opportunities' tab (page title
              'Opportunities - ORIN') so the recommended opportunity listing
              can be reopened and its detail view opened for inspection.
76            # Switch to tab DCA7
77            page = context.pages[-1]  # switch to most recently active tab
78
79            # -> Click the 'Startup Mentorship Track' card (the card
              titled 'Startup Mentorship Track' for Y Combinator) to open
              the opportunity's detail view inside ORIN.
80            # Y
81            elem = page.get_by_text('Y', exact=True)
82            await elem.click(timeout=10000)
83
84            # -> Click the 'Startup Mentorship Track' card (the card
              container/title on the Opportunities page) to open its
              internal opportunity detail view for inspection.
85            # Y
86            elem = page.get_by_text('Y', exact=True)
87            await elem.click(timeout=10000)
88
89            # -> Click the 'Startup Mentorship Track' opportunity card
              (the card area that shows the title 'Startup Mentorship
              Track') to open its internal detail view for inspection.
90            # Y
```

```
            elem = page.get_by_text('Y', exact=True)
            await elem.click(timeout=10000)

            # --> Assertions to verify final state

            # --> Verify the opportunity details are displayed
            # Assert: Expected URL to navigate to an opportunity detail
            page.
            await expect(page).to_have_url(re.compile("^https://orin-three.
            vercel.app/opportunities/.+"), timeout=15000), "Expected URL
            to navigate to an opportunity detail page."
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST FAILURE Opening an opportunity detail view did not work — the internal opportunity detail view could not be reached from the Opportunities listing. Observations: - Clicking the 'Apply' link for 'Startup Mentorship Track' opened a new tab to https://www.ycombinator.com/ (external site). - Clicking the 'Startup Mentorship Track' card/title did not open an internal detail view; the Opportunities list remained visible.

Cause

The hosted app appears to have the Startup Mentorship Track listing wired only to an external outbound Apply link (to ycombinator.com) and not to an internal detail route. In addition, the detail route may be missing or not registered in the SPA router/hosting rewrites, so clicking the card/title either does nothing or keeps the user on the listing page instead of rendering an internal opportunity detail view.

Fix

Update the Opportunities listing so the card/title routes to an internal opportunity detail page (for example, /opportunities/:id) and ensure a corresponding detail route/component exists and is reachable from the client-side router. If the intent is to keep an external Apply link, separate it from the card navigation: make the card/title clickable to the internal detail view, and keep Apply as an external CTA. Also verify any hosting rewrites are configured so direct navigation to the internal detail route is served by the app rather than falling back to the listing page.

AI Career Coach and Learning Paths: AI coach shows generated coaching notes

ATTRIBUTES

Status	Failed
Priority	Medium
Description	A user opens coaching notes and reviews the generated advice. The page should present clear notes and practical recommendations the user can act on.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714715911518//tmp/3e43fc5f-2349-4fb3-a91f-71192a8534ad/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom
        arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability
        budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for)
        inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the top navigation to open the
        sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator
        ("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link',
        name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the 'Email address' field with the test email, fill
        the 'Password' field with the test password, then click the
        'Sign in' button to submit the form.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the 'Email address' field with the test email, fill
              the 'Password' field with the test password, then click the
              'Sign in' button to submit the form.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the 'Email address' field with the test email, fill
              the 'Password' field with the test password, then click the
              'Sign in' button to submit the form.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the "AI Chat" navigation item in the left sidebar
              to open the AI Chat page.
66            # AI Chat link
67            elem = page.get_by_role('link', name='AI Chat', exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Open the activity/notes panel by clicking the 'Toggle
              activity panel' button to reveal coaching notes or actionable
              recommendations.
71            # Toggle activity panel button
72            elem = page.get_by_role('button', name='Toggle activity
              panel', exact=True)
73            await elem.click(timeout=10000)
74
75            # -> Type a request into the 'Ask Orin anything...' input
              asking Orin to generate coaching notes and practical,
              actionable recommendations, then press Enter to send the
              message.
76            # Ask Orin anything... text area
77            elem = page.get_by_placeholder('Ask Orin anything...',
              exact=True)
78            await elem.wait_for(state="visible", timeout=10000)
79            await elem.fill("Please generate clear coaching notes about my
              professional profile and 5 practical, actionable
              recommendations I can follow in the next 30 days. Present
              concise notes and prioritized next steps.")
80
81            # -> Click the visible 'Retry' button in the chat (label:
              'Retry') to resend the request and wait for the AI response.
82            # Retry button
83            elem = page.get_by_role('button', name='Retry', exact=True)
84            await elem.click(timeout=10000)
85
86            # -> Click the 'Retry' button in the chat to resend the prompt
              and wait for the AI response, then verify that concise
              coaching notes and five practical, actionable recommendations
              appear in the chat or activity panel.
87            # Retry button
88            elem = page.get_by_role('button', name='Retry', exact=True)
89            await elem.click(timeout=10000)
```

```
90
91          # -> Click the 'Retry' button in the chat to resend the prompt
            and then wait for the AI response to appear.
92          # Retry button
93          elem = page.get_by_role('button', name='Retry', exact=True)
94          await elem.click(timeout=10000)
95
96          # -> Start a new conversation by clicking the 'New
            conversation / Toggle conversation history' control, then
            paste the coaching-notes prompt into the 'Ask Orin anything...
            ' textbox and press Enter to submit.
97          # Toggle conversation history button
98          elem = page.get_by_role('button', name='Toggle conversation
            history', exact=True)
99          await elem.click(timeout=10000)
100
101         # -> Start a new conversation by clicking the 'New
            conversation / Toggle conversation history' control, then
            paste the coaching-notes prompt into the 'Ask Orin anything...
            ' textbox and press Enter to submit.
102         # Ask Orin anything... text area
103         elem = page.get_by_placeholder('Ask Orin anything...',
            exact=True)
104         await elem.wait_for(state="visible", timeout=10000)
105         await elem.fill("Please generate clear coaching notes about my
            professional profile and 5 practical, actionable
            recommendations I can follow in the next 30 days. Present
            concise notes and prioritized next steps.")
106
107         # --> Assertions to verify final state
108
109         # --> Verify generated coaching notes are displayed
110         # Assert: Expected the chat content to contain generated
            coaching notes.
111         await expect(page.locator("xpath=/html/body/div[2]/main/div/
            div/div[2]/div[1]").nth(0)).to_contain_text("coaching notes",
            timeout=15000), "Expected the chat content to contain
            generated coaching notes."
112         # Assert: Expected the chat content to include five practical,
            actionable recommendations.
113         await expect(page.locator("xpath=/html/body/div[2]/main/div/
            div/div[2]/div[1]").nth(0)).to_contain_text("5 practical,
            actionable recommendations", timeout=15000), "Expected the
            chat content to include five practical, actionable
            recommendations."
114
115         # --> Verify actionable recommendations are displayed
116         # Assert: Expected actionable recommendations to be displayed
            in the activity panel.
117         await expect(page.locator("xpath=/html/body/div[2]/main/div/
            div/div[2]/div[1]").nth(0)).to_contain_text("actionable
            recommendations", timeout=15000), "Expected actionable
            recommendations to be displayed in the activity panel."
118         # Assert: Expected actionable recommendations to include '5
            practical, actionable recommendations' in the activity panel.
119         await expect(page.locator("xpath=/html/body/div[2]/main/div/
            div/div[2]/div[1]").nth(0)).to_contain_text("5 practical,
            actionable recommendations", timeout=15000), "Expected
            actionable recommendations to include '5 practical, actionable
```

```
            recommendations' in the activity panel."
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST FAILURE The AI chat feature failed to generate coaching notes and actionable recommendations — the chat returned an error message instead of a valid response. Observations: - The chat displays the message 'Something went wrong. Please try again.' after the prompt was submitted. - Retry was clicked three times and a new conversation was started and retried, but the error persisted. - The activity panel remains empty and no coaching notes or recommendations are present.

Cause

The failure is likely on the hosting/backend side of the AI chat integration: the production API route that generates coaching notes is probably misconfigured, missing required environment variables, hitting a provider/auth/rate-limit error, or returning an unexpected response shape. That would make the frontend show the generic 'Something went wrong. Please try again.' message and leave the activity panel empty even after retries.

Fix

Check the hosted AI/chat backend logs and verify the deployed environment variables for the model provider (API key, base URL, org/project, model name) are present and correct in production. If the endpoint is throwing a 4xx/5xx or returning malformed data, fix the server-side handler to return a valid JSON response matching the frontend contract, add proper timeout/retry handling, and ensure the route is deployed with the correct runtime permissions/cors settings. If the issue is due to a broken or rate-limited model integration, rotate credentials or switch to a working model/provider and redeploy.

Verified Profile and Public Presence: View public profile as a recruiter

ATTRIBUTES

Status	Failed
Priority	High
Description	A recruiter or visitor opens a public profile page and reviews verified proof cards. The page should present source-linked achievements that are visible without signing in.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714410663238//tmp/ddcd0549-d18a-42b2-8b4e-5a80ff1deafa/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the site's public profile demo page (the Public Profile / Test page) so the profile's proof cards can be inspected.
        await page.goto("https://orin-three.vercel.app/test")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # --> Assertions to verify final state
        # Assert: Verify verified proof cards are displayed
```

```
            assert False, "Expected: Verify verified proof cards are
            displayed (could not be verified on the page)"
            # Assert: Verify source-linked achievements are shown
            assert False, "Expected: Verify source-linked achievements are
            shown (could not be verified on the page)"
            await asyncio.sleep(5)

        finally:
            if context:
                await context.close()
            if browser:
                await browser.close()
            if pw:
                await pw.stop()

    asyncio.run(run_test())

```

Error

TEST FAILURE The public profile page is accessible without signing in but does not present any verified proof cards or source-linked achievements for review. The expected recruiter/visitor experience (visible, source-linked achievement/proof cards on the public profile) is not available on this profile. Observations: - The profile header is visible and the page displays the message: 'No public proofs yet'. - The metrics section shows 0 Verified Proofs, 0 Skills Proven, 0 Profile Views, and 0 Connected Sources. - No proof cards, external source links, or achievement badges are present on the page for a recruiter to inspect.

Cause

The hosting-side failure is likely due to the production deployment serving an empty or misconfigured profile dataset, so the public profile page renders only the fallback state. This can happen if the app is pointed at the wrong database/environment, the proof/achievement records were not migrated or seeded in production, the public-profile API is not returning linked proof data because of a backend/auth misconfiguration, or the server-rendered page is cached/stale and never receives the verified proofs needed to display recruiter-visible cards.

Fix

Verify the production data source and seeded content for the public profile route: ensure the deployed environment is connected to the correct database/backend, the profile being tested has public proof records and connected sources, and the API/SSR logic actually fetches and renders verified proof cards on the public profile page. If the issue is missing production data, seed or migrate public proofs/achievements for that account. If it is a deployment/config issue, check environment variables, auth/DB permissions, and caching so the public profile endpoint returns the expected proof objects and source links instead of the fallback 'No public proofs yet' state.

Proof Cards: View a public profile with verified proof cards

ATTRIBUTES

Status	Failed
Priority	High
Description	A recruiter or visitor opens a public profile and sees the user's proof-based presentation. The test confirms verified proof cards and source-linked achievements are visible on the public page.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/17817143807006//tmp/07861799-fa3a-4d39-aa07-7ea24b582eb0/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Open the public profile page titled 'Public Profile' (URL: /test) and load it so the page can be inspected for verified proof cards and source-linked achievements.
        await page.goto("https://orin-three.vercel.app/test")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Scroll down the Public Profile page and search for the text 'No public proofs yet' or visible proof cards/badges to
```

```
            confirm whether verified proof cards and source-linked
            achievements are present.
51          await page.mouse.wheel(0, 300)
52
53          # --> Assertions to verify final state
54          # Assert: Verify verified proof cards are displayed
55          assert False, "Expected: Verify verified proof cards are
            displayed (could not be verified on the page)"
56          # Assert: Verify source-linked achievements are displayed
57          assert False, "Expected: Verify source-linked achievements are
            displayed (could not be verified on the page)"
58          await asyncio.sleep(5)
59
60      finally:
61          if context:
62              await context.close()
63          if browser:
64              await browser.close()
65          if pw:
66              await pw.stop()
67
68  asyncio.run(run_test())
69
```

Error

TEST FAILURE Verified proof cards and source-linked achievements are not displayed on the public profile page. Observations: - The public profile page shows the message 'No public proofs yet'. - The profile counters show zeros for Verified Proofs, Skills Proven, Profile Views, and Connected Sources. - No verified proof cards or source-linked achievement badges are visible on the page.

Cause

The page appears to be rendering successfully but with zeroed counters and no proof cards, which suggests the production deployment is not receiving the expected profile data from the backend. Common hosting-side causes include incorrect or missing environment variables, a misconfigured API URL pointing to an empty staging/local database, failed server-side fetches due to auth/permission issues, or caching/ISR serving a default empty state instead of live profile content.

Fix

The hosting environment is likely serving a public profile page backed by empty or mismatched production data, or it is not connecting to the correct backend services (database/auth/storage) in production. Verify the deployed environment variables and API endpoints on Vercel, ensure the production database contains the verified proofs and connected-source records for the tested profile, and confirm the serverless functions/SSR fetches are not failing or being blocked by permissions. Also check that any build-time or runtime data fetching isn’t using stale cache or an incorrect profile identifier, and redeploy after fixing the production data source and access permissions.

Opportunity Matching and Search: Track an opportunity application

ATTRIBUTES

Status	Blocked
Priority	Low
Description	A user updates the progress of an opportunity they are pursuing. The new application status should be saved and reflected in the tracking view.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/44f89458-e0a1-7045-eb2f-76be48dd8e30/1781714872537694//tmp/cc135bf1-0a84-461d-8006-02d17077e4e7/result.webm

</> Test Code

```
import asyncio
import re
from playwright import async_api
from playwright.async_api import expect

async def run_test():
    pw = None
    browser = None
    context = None

    try:
        # Start a Playwright session in asynchronous mode
        pw = await async_api.async_playwright().start()

        # Launch a Chromium browser in headless mode with custom arguments
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                "--window-size=1280,720",
                "--disable-dev-shm-usage",
                "--ipc=host",
                "--single-process"
            ],
        )

        # Create a new browser context (like an incognito window)
        context = await browser.new_context()
        # Wider default timeout to match the agent's DOM-stability budget;
        # auto-waiting Playwright APIs (expect, locator.wait_for) inherit this.
        context.set_default_timeout(15000)

        # Open a new page in the browser context
        page = await context.new_page()

        # Interact with the page elements to simulate user flow
        # -> navigate
        await page.goto("https://orin-three.vercel.app/")
        try:
            await page.wait_for_load_state("domcontentloaded", timeout=5000)
        except Exception:
            pass

        # -> Click the 'Log in' link in the top navigation to open the sign-in page.
        # Log in link
        elem = page.get_by_text('ORIN', exact=True).locator("xpath=ancestor-or-self::*[.//a][1]").get_by_role('link', name='Log in', exact=True)
        await elem.click(timeout=10000)

        # -> Fill the email field with 'test@email.com', fill the password field with 'Test@123', and click the 'Sign in' button to authenticate.
        # you@example.com email field
```

```
50            elem = page.locator('[id="email"]')
51            await elem.wait_for(state="visible", timeout=10000)
52            await elem.fill("test@email.com")
53
54            # -> Fill the email field with 'test@email.com', fill the
              password field with 'Test@123', and click the 'Sign in' button
              to authenticate.
55            # Enter your password password field
56            elem = page.locator('[id="password"]')
57            await elem.wait_for(state="visible", timeout=10000)
58            await elem.fill("Test@123")
59
60            # -> Fill the email field with 'test@email.com', fill the
              password field with 'Test@123', and click the 'Sign in' button
              to authenticate.
61            # Sign in button
62            elem = page.get_by_role('button', name='Sign in', exact=True)
63            await elem.click(timeout=10000)
64
65            # -> Click the 'Opportunities' link in the left sidebar to
              open the Opportunities page.
66            # Opportunities link
67            elem = page.get_by_role('link', name='Opportunities',
              exact=True)
68            await elem.click(timeout=10000)
69
70            # -> Open the opportunity tracking view by clicking the
              'Apply' button on the 'ML Engineering Intern' card (the
              'Apply' button labeled 'Apply' on that card).
71            # Apply link
72            elem = page.get_by_text('OPInternshipML Engineering
              InternOpenAI', exact=True).locator("xpath=ancestor-or-self::*
              [.//a][1]").get_by_role('link', name='Apply', exact=True)
73            await elem.click(timeout=10000)
74
75            # -> Switch to the 'Opportunities' tab and open the ML
              Engineering Intern opportunity's tracking/detail view by
              clicking the opportunity card or its 'Apply'/'View' link in
              the Opportunities listing.
76            # Switch to tab 8741
77            page = context.pages[-1]  # switch to most recently active tab
78
79            # -> Click the 'Apply' button on the 'ML Engineering Intern'
              card to open the opportunity's tracking/detail view (or
              confirm that it opens an external page), then verify whether
              an in-app tracking view is available.
80            # Apply link
81            elem = page.get_by_text('OPInternshipML Engineering
              InternOpenAI', exact=True).locator("xpath=ancestor-or-self::*
              [.//a][1]").get_by_role('link', name='Apply', exact=True)
82            await elem.click(timeout=10000)
83
84            # -> Switch to the 'Opportunities' tab and open the 'ML
              Engineering Intern' opportunity card or its in-app 'View'/
              'Tracking' link to access the opportunity's tracking/detail
              view.
85            # Switch to tab 8741
86            page = context.pages[-1]  # switch to most recently active tab
87
```

```
88            # -> Click the visible 'ML Engineering Intern' job card (the
              job card area showing the title) to try to open its in-app
              tracking/detail view.
89            # OP
90            elem = page.get_by_text('OP', exact=True)
91            await elem.click(timeout=10000)
92
93            # -> Click the 'ML Engineering Intern' job card on the
              Opportunities page to open its detail/tracking view (click the
              card area or title rather than the 'Apply' button).
94            # OP
95            elem = page.get_by_text('OP', exact=True)
96            await elem.click(timeout=10000)
97
98            # -> Click the 'Save' button on the ML Engineering Intern
              opportunity card to see if it opens an in-app detail or
              reveals tracking/progress options.
99            # Save button
100           elem = page.get_by_text('OPInternshipML Engineering
              InternOpenAI', exact=True).locator("xpath=ancestor-or-self::*
              [.//button][1]").get_by_role('button', name='Save', exact=True)
101           await elem.click(timeout=10000)
102
103           # --> Assertions to verify final state
104           # Assert: Verify the updated application progress is displayed
105           assert False, "Expected: Verify the updated application
              progress is displayed (could not be verified on the page)"
106
107           # --> Test blocked by environment/access constraints during
              agent run
108           # Reason: TEST BLOCKED The tracking view could not be reached
              — the in-app opportunity tracking/detail view is not
              accessible from the Opportunities listing, blocking the
              ability to update and verify application progress.
              Observations: - Clicking the 'Apply' button for the ML
              Engineering Intern opened an external careers page in a new
              tab which landed on a Cloudflare human verification page,
              preventing a...
109           raise AssertionError("Test blocked during agent run: " + "TEST
              BLOCKED The tracking view could not be reached \u2014 the
              in-app opportunity tracking/detail view is not accessible from
              the Opportunities listing, blocking the ability to update and
              verify application progress. Observations: - Clicking the
              'Apply' button for the ML Engineering Intern opened an
              external careers page in a new tab which landed on a
              Cloudflare human verification page, preventing a..." + " — the
              exported script cannot reproduce a PASS in this environment.")
110           await asyncio.sleep(5)
111
112       finally:
113           if context:
114               await context.close()
115           if browser:
116               await browser.close()
117           if pw:
118               await pw.stop()
119
120   asyncio.run(run_test())
121
```

Error

TEST BLOCKED The tracking view could not be reached — the in-app opportunity tracking/detail view is not accessible from the Opportunities listing, blocking the ability to update and verify application progress. Observations: - Clicking the 'Apply' button for the ML Engineering Intern opened an external careers page in a new tab which landed on a Cloudflare human verification page, preventing access to the target flow. - Clicking the ML Engineering Intern card area directly did not open any in-app detail or tracking view despite multiple attempts; the Opportunities listing remained visible.